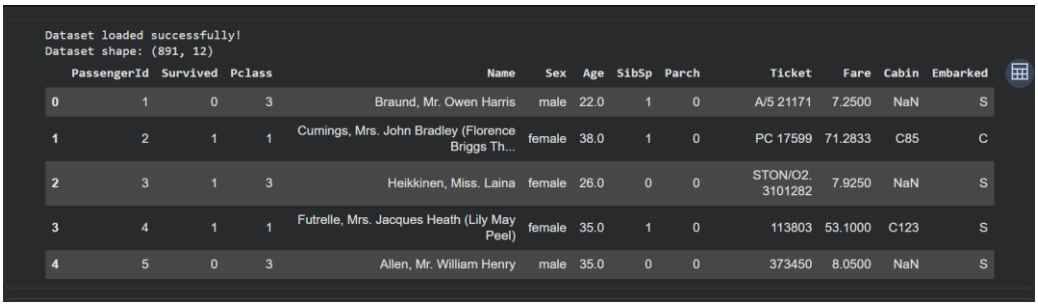
CLA 4: ML Model Deployment Strategies and Model Serving Using FastAPI

Dataset: Titanic – Machine Learning from Disaster (Kaggle)

Dataset Link: https://www.kaggle.com/competitions/titanic/data

1. Objective

The objective of this assignment is to develop a machine learning classification model using the Titanic dataset and demonstrate different inference strategies, including online inference, batch inference, and streaming inference. The trained model is deployed through a REST API using FastAPI.



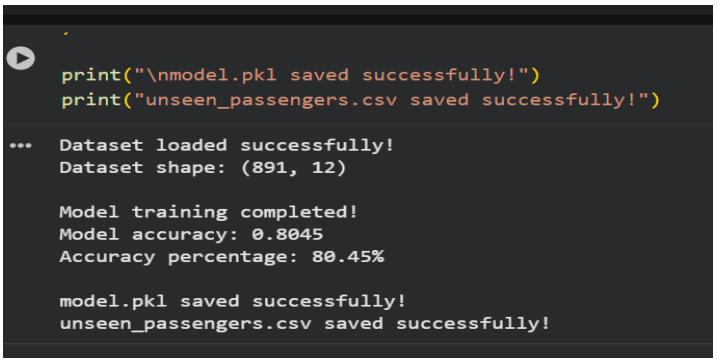
```
Dataset loaded successfully!
Dataset shape: (891, 12)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

2. Model Training

The Titanic dataset (train.csv) was loaded using Pandas. Data preprocessing was performed to handle missing values, scale numerical features, and encode categorical features. Logistic Regression was used as the classification algorithm.

The dataset was divided into training and testing sets using an 80:20 ratio. The model was evaluated using accuracy, and the complete machine learning pipeline was saved using Joblib as model.pkl.



```
print("\nmodel.pkl saved successfully!")
print("unseen_passengers.csv saved successfully!")
```

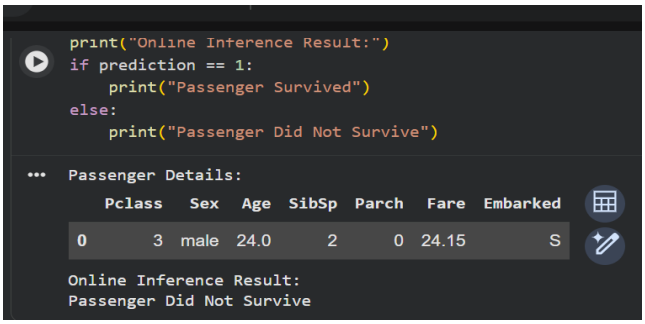
```
Dataset loaded successfully!
Dataset shape: (891, 12)

Model training completed!
Model accuracy: 0.8045
Accuracy percentage: 80.45%

model.pkl saved successfully!
unseen_passengers.csv saved successfully!
```

3. Online Inference

Online inference was performed by loading the saved model and selecting one passenger record from the held-out test dataset. The model predicted whether the passenger would survive or not survive.



```
print("Online Inference Result:")
if prediction == 1:
    print("Passenger Survived")
else:
    print("Passenger Did Not Survive")
```

```
Passenger Details:
```

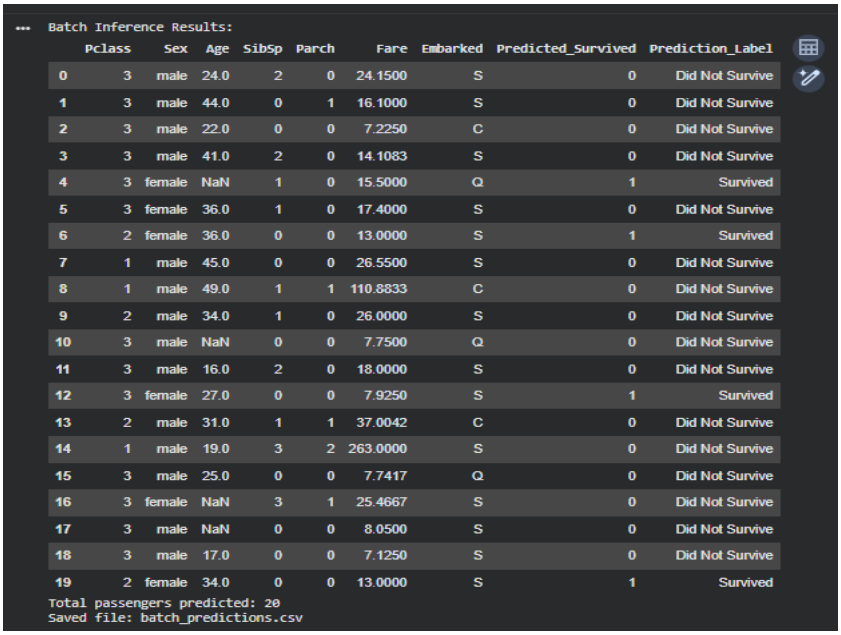
	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	male	24.0	2	0	24.15	S

```
Online Inference Result:
Passenger Did Not Survive
```

4. Batch Inference

Batch inference was performed by selecting 20 passenger records from the held-out test dataset. The saved model generated predictions for all 20 passengers together. The results were stored in batch_predictions.csv.

Result: Predictions were generated for 20 passengers and saved in a CSV file.



Batch Inference Results:

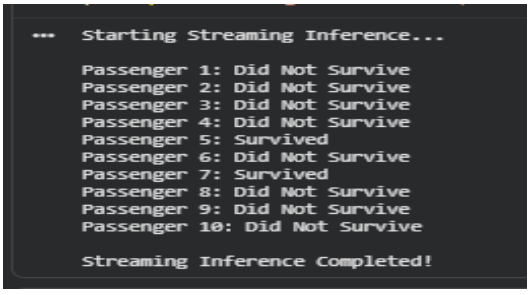
	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Predicted_Survived	Prediction_Label
0	3	male	24.0	2	0	24.1500	S	0	Did Not Survive
1	3	male	44.0	0	1	16.1000	S	0	Did Not Survive
2	3	male	22.0	0	0	7.2250	C	0	Did Not Survive
3	3	male	41.0	2	0	14.1083	S	0	Did Not Survive
4	3	female	NaN	1	0	15.5000	Q	1	Survived
5	3	female	36.0	1	0	17.4000	S	0	Did Not Survive
6	2	female	36.0	0	0	13.0000	S	1	Survived
7	1	male	45.0	0	0	26.5500	S	0	Did Not Survive
8	1	male	49.0	1	1	110.8833	C	0	Did Not Survive
9	2	male	34.0	1	0	26.0000	S	0	Did Not Survive
10	3	male	NaN	0	0	7.7500	Q	0	Did Not Survive
11	3	male	16.0	2	0	18.0000	S	0	Did Not Survive
12	3	female	27.0	0	0	7.9250	S	1	Survived
13	2	male	31.0	1	1	37.0042	C	0	Did Not Survive
14	1	male	19.0	3	2	263.0000	S	0	Did Not Survive
15	3	male	25.0	0	0	7.7417	Q	0	Did Not Survive
16	3	female	NaN	3	1	25.4667	S	0	Did Not Survive
17	3	male	NaN	0	0	8.0500	S	0	Did Not Survive
18	3	male	17.0	0	0	7.1250	S	0	Did Not Survive
19	2	female	34.0	0	0	13.0000	S	1	Survived

Total passengers predicted: 20
Saved file: batch_predictions.csv

5. Streaming Inference

Streaming inference was simulated using 10 passenger records. Each passenger record was processed individually, with a one-second delay between records. The prediction for each incoming passenger was displayed on the screen.

Result: The streaming inference process displayed predictions for 10 passengers sequentially.



```
Starting Streaming Inference...

Passenger 1: Did Not Survive
Passenger 2: Did Not Survive
Passenger 3: Did Not Survive
Passenger 4: Did Not Survive
Passenger 5: Survived
Passenger 6: Did Not Survive
Passenger 7: Survived
Passenger 8: Did Not Survive
Passenger 9: Did Not Survive
Passenger 10: Did Not Survive

Streaming Inference Completed!
```

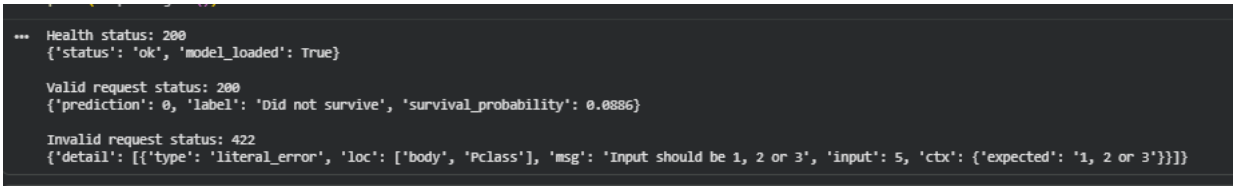
6. Model Serving Using FastAPI

FastAPI was used to expose the trained machine learning model through REST API endpoints. Pydantic was used to validate the incoming passenger data.

The following endpoints were implemented:

- **GET /health:** Checks the API status and confirms that the model is loaded.
- **POST /predict:** Accepts passenger details and returns the predicted survival status and survival probability.

The API was tested using FastAPI's TestClient. A valid passenger request returned HTTP 200, while an invalid request returned HTTP 422 due to data validation.



7. Observations

7.1 How does Online Inference differ from Batch Inference?

Online inference predicts one record at a time and returns the result immediately. Batch inference processes multiple records together and generates predictions for the entire group. Online inference is useful for individual requests, whereas batch inference is suitable for processing larger datasets.

7.2 What is the difference between Streaming Inference and Batch Inference?

Streaming inference processes records individually as they arrive, usually continuously or in near real time. Batch inference collects a group of records and processes them together. Streaming inference is useful for continuously arriving data, while batch inference is useful for scheduled processing.

7.3 Why is FastAPI used for model serving?

FastAPI is used to expose a trained machine learning model through REST API endpoints. It supports Pydantic-based data validation, JSON responses, and automatic interactive API documentation through Swagger UI.

7.4 Why should the same trained model be reused for all inference strategies?

The same saved model should be reused to maintain consistency in predictions and ensure a fair comparison between online, batch, and streaming inference. It also avoids unnecessary retraining and ensures that the deployed API uses the same model pipeline.

8. Conclusion

In this assignment, a Logistic Regression classification model was trained using the Titanic dataset. The complete preprocessing and prediction pipeline was saved using Joblib. Online, batch, and streaming inference were demonstrated using passenger records excluded from model training. FastAPI was used to create REST endpoints for health checking and survival prediction, and input validation was tested using valid and invalid requests.

This assignment demonstrates the process of training, evaluating, saving, and deploying a machine learning model for practical use.